TARGET SHIP DATE: {2026-11-10}



(Team Flag)

TNPG

Spin The Wheel

Roster + **Roles**: Matthew Chen – Project Manager, Scripter Sakina Raza – CSS, Design Doc Caroline Tham – Jinja/HTML, Scripter Efaz Khan – Scribe, SQL	Matthew Chen, Sakina Raza, Caroline Tham, Efaz Khan PM: Matthew Chen TNPG: Spin The Wheel Software Development Pr0 - Scenario Sigma Target Ship Date: 2026-11-09 Time Spent: 4.5 hrs Click here!

Team emails:

Name	Email	Primary Role	Secondary Role
Matthew Chen	matthewc800@nycstudents.net	Project Manager	Scripter
Sakina Raza	sakinar3@nycstudents.net	Flask	Design Doc
Caroline Tham	carolinet19@nycstudents.net	CSS	HTML
Efaz Khan	efazk3@nycstudents.net	Scribe	SQL

Summary:

We are building a collaborative storytelling website where registered users start stories and add to each other's, seeing only the latest update and adding to each story only once.

Team Task:

Our team has been contracted to create a storytelling website. Users register/log in, and a logged-in user can start a new story (a title plus any amount of opening text) or add to an existing story. When adding, the user sees only the latest update, never the whole story, and writes the next piece. A user can add to a given story only once. Every story a user has contributed to is readable in full on their homepage, which is the landing page after login.

Problem Being Solved:

Writing a story together is rare, as usually one person controls the plot. In our project, we have each story writer work from only the last section before it, so that the story goes in unexpected directions. This allows for collaborative storytelling where everyone contributes something unique.

Target Users

- Writers who want to participate in a quick collaborative story
- Readers who enjoy reading stories with unexpected twists and turns

Why This Project Matters:

This project creates a website with a system to create collaborative story-telling, as opposed to the traditional single author narrative arc. This system matters as it allows a new form of writing

with collective narratives that can be influenced and changed by twists and unexpected edits by peers. It is also an important model of full-stack web engineering, as it combines user-authentification, database constraints, and data flow orchestration between Flask, SQLite, and Jinja templates.

Minimum Viable Product (MVP)

Core Features:

1. Register and log in using Flask sessions
2. Start a new story with a title and opening text
3. Browse stories you haven't contributed to and add to one, seeing only the latest update, once per story
4. Profile page listing every story you've contributed to, with the full text readable

Stretch Features:

1. Show contributor usernames in order for each story
2. Being able to sort/search through stories on /browse
3. Making the website look really nice (through CSS)

Explicit Non-Goals/Features Intentionally Excluded:

1. Editing/deleting a contribution thats already submitted
2. Comments/likes/following users or stories
3. Password reset

Technology Stack

Layer	Selected Tool
Backend Framework	Flask
Frontend Framework	none
Database	SQLite
Authentication	Flask Sessions

Why This Stack was Chosen: This stack is fixed/given for Project0.

Team Ownership Plan

Team Member	Primary Ownership	Secondary Ownership	Specific Deliverables
Matthew Chen	Authentication, Story management.	CSS/static	register/login/logout with sessions; story rules (create story, latest update, once per story)
Sakina Raza	CSS, HTML templates	Flask server	html page templates; css stylesheets; design.pdf
Caroline Tham	Flask server (serve.py)	Story management, Database.	all routes wired together; session checks on protected pages
Efaz Khan	SQL Database	HTML templates, Authentication	create the 3 tables with constraints; helper functions for users, stories, contributions;

Group Tasks:

Flask server (serve.py) - Caroline/Sakina

- Overarching file that hosts and connects all the components

Authentication - Matthew/Efaz

- Handles login/logout/registration

Database management (SQL) - Efaz/Caroline

- Stores data such as user accounts and stories in a database

Story management - Matthew/Caroline

- Handles the different stories and how to add to them

HTML templates - Sakina/Efaz

- Provides the skeleton for the web pages

CSS files/static files - Sakina/Matthew

- Styles the web page/provides helpful graphics

Components

‘/’ → (Landing Page)

- Welcome page
- Buttons to register or login

‘/register’

- Username and password text boxes
- check Users table for unique username (inserts it if unique)
- After registered goes to /profile

‘/login’

- Username and password text boxes
- check Users table for correct user/pass
- After logged in goes to /profile

‘/profile’

- For logged in people
- List of stories the user has contributed to, with the full text readable
- Buttons to start a new story, add to one, or logout

‘/new’

- Title text box and Opening text box
- Submit button: inserts to Stories table, adds first row into Contributions table, goes to /profile

‘/browse’

- List of story titles the user has not contributed to (no story text shown)
- Each title links to /add/<story_id>

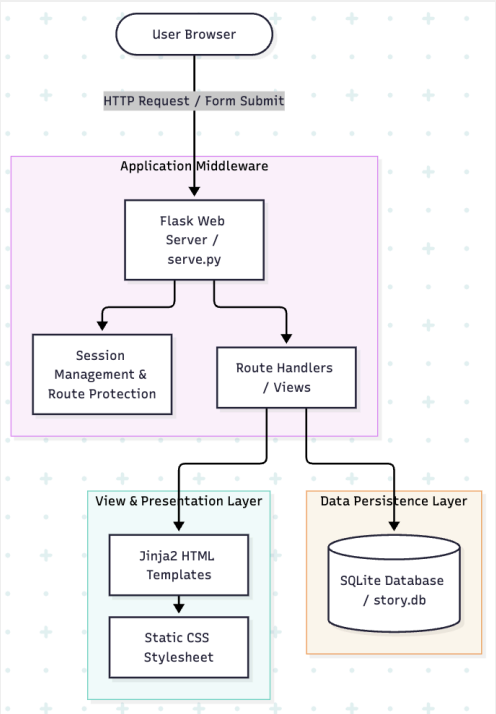
‘/logout’

- Redirects back to ‘/profile’ via button click
- Clears Flask session → session.clear() → redirects to home

‘/add/<story_id>’

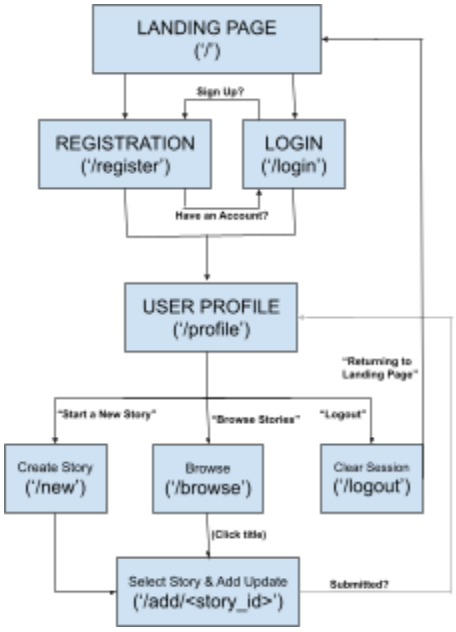
- Shows the latest update of the story
- Text box and submit button
- Checks Contributions table: if user had added to this story, redirect with error message
- After submitting, inserts into Contributions table and goes to ‘/profile’

Component Relationship Map



(Mermaid diagrams)

Site Map



Key User Stories

- As a registered user, I want to start a story with a title and opening text so that other writers can continue it.
- As a writer, I want to see only the latest update before adding my piece so that the story can surprise me.
- As a contributor, I want to read every story I've helped write in full on my profile so that I can see how it turned out.

Databases

Users:

- user_id is a PRIMARY KEY
- username is UNIQUE

user_id	username	password
1	user1	pass1
2	user2	pass2

Stories:

- user_id is the user_id of whoever started the story
- story_id is a PRIMARY KEY to avoid same story being made again

story_id	title	user_id
1	title1	1
2	title2	2

Contributions:

- contribution_id is a PRIMARY KEY
- user_id is the user_id of whoever created the contribution
- story_id is the story_id of the story that is being contributed too

- when a user makes a story, their story is saved here as the first contribution, so the creator can't add to it again either (a user can only have one contribution per story, so they can't add to the same story twice)

contribution_id	story_id	user_id	body
1	1	1	this is whats in story 1
2	1	2	adding this to story 1

Testing Plan

Database

- The 3 tables are created the first time the app runs, and running it again doesn't wipe or duplicate them
- Inserting a second contribution with the same story_id and user_id is rejected (tests the UNIQUE rule)
- Inserting a user with a username that already exists is rejected
- Helper functions return the right rows (get a user, get a story, get the latest contribution)

Authentication

- Register with a new username: account is created and you land on /profile
- Register with a taken username: shows an error and creates nothing
- Register when the password and verify password don't match: shows an error
- Login with the right username and password: lands on /profile
- Login with a wrong password or unknown username: shows an error
- Logout clears the session, so going back to /profile sends you to /login
- Visiting /profile, /new, /browse, or /add/<story_id> while logged out redirects to /login

Story management

- Creating a story adds one row to Stories and one row to Contributions (the opening text)
- The latest update shown on /add/<story_id> is the contribution with the highest contribution_id
- After you contribute to a story, it no longer appears on /browse
- Adding to the same story a second time is blocked with an error message
- The creator can't add to their own story again
- Visiting /add/<story_id> with an id that doesn't exist doesn't crash the site

- Empty title or empty text is rejected

Flask routes

- Every route loads without errors and links go where the site map says
- Form submissions (register, login, /new, /add) save right data and redirects to right page
- Using two accounts, a story started by one user can be continued by the other and then shows up on both profiles

Templates and CSS

- Every page renders and every link and button works
- With the CSS file removed, every page still works and is usable
- Profile shows the full text of every story the user has contributed to
- Pages look right at a smaller window size (quick check)

Completion

Timeline/Internal Deadlines:

- **Phase 1: 11/5/26**
  - Repo organization, submodule setup, flag initialization
- **Phase 2: 11/8/26**
  - Design Doc initialization, drafting initial specs, doc hardcopies
- **Phase 3: TBD**
  - Basic webpage hosting, setting up serve.py, basic template structure
- **Phase 4: TBD**
  - Revising design doc, archiving design0.pdf, design.md creation in repo root
- **Phase 5: TBD**
  - SQLite database implementation(users, stories, contributions), keys, constraints
- **Phase 6: TBD**
  - User authentication(login, register, logout routes), Flask session management
- **Phase 7: TBD**
  - Core story management implementation(/new, /browse, /add/story_id routes)
- **Phase 8: TBD**
  - Front-end styling and integration, modular CSS (if styles remove it still works)
- **Phase 9: TBD**
  - Final doc, “Install Guides” and “Launch Codes” in readme, requirements.txt done
- **Phase 10: TBD**
  - Final testing, readme review logging, devlog audit

Completion Criteria:

- User can register and log in using Flask sessions
- User can start a new story with a title and opening text
- User can browse stories they haven't contributed to and add to one, seeing only the latest update, once per story
- User can see profile page listing every story you've contributed to, with the full text readable
- Everything in the testing plan passes

Open Questions:

- Who is Clyde Thuffy Sinclair?

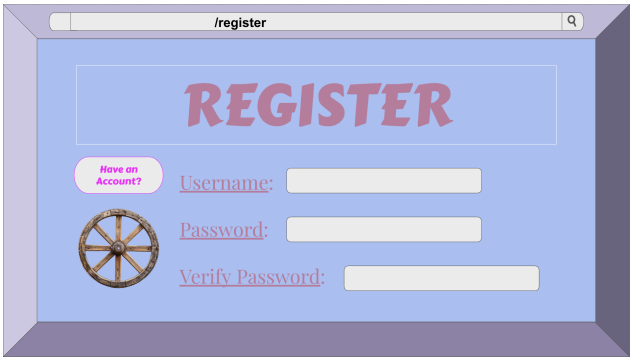
Appendix/Other

This is our front-end site map: Click here to view the Site Map!

Landing Page



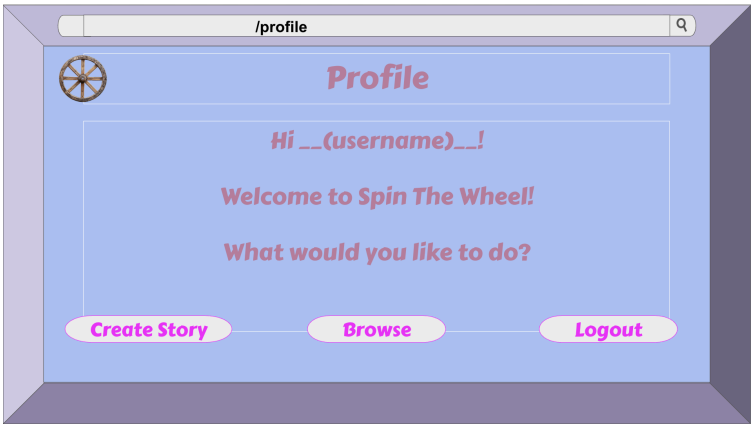
Registration



Login



User Profile



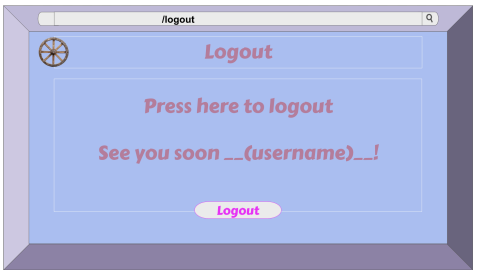
Create Story



Browse



Clear Session



Select Story & Add Update



Structure Tree

```
app/
    serve.py
    static/
        style.css
    templates/
        index.html       # '/' (Landing page with login/register links)
        register.html    # '/register' (Registration form)
        login.html       # '/login' (Login form)
        profile.html     # '/profile' (User dashboard showing contributed stories & actions)
        new.html         # '/new' (Form to start a new story)
        browse.html      # '/browse' (List of stories user has NOT contributed to)
        add.html         # '/add/<story_id>' (View latest update & submit continuation)
design.md
design.pdf
devlog.txt
flag.jpg
README.md
requirements.txt
```